Practical Digital Systems Design

Homework 2:

Voting Circuit &

Median Circuit Design

Name: 尋以諾

Student ID: B123015021

I. Design Principle

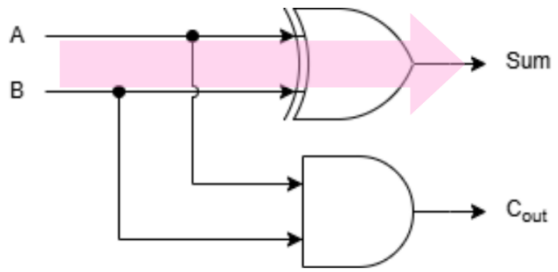
1. Voting Circuit

The voting circuit determines which 3-bit one-hot number appears most often among five inputs and outputs that number along with a 3-bit binary count of its occurrences.

Input	Output
5 same	Output 5-same value
4 same + 1 diff	Output the 4-same value
3 same + 2 same	Output the 3-same value
3 same + 2 diff	Output the 3-same value
2 same + 2 same + 1 diff	Output any 2-same value

By the property of one-hot encoding, only one bit in each input is '1' while all others are '0'. Therefore, by adding the bits in the same position across the five inputs, we can determine how many times each position was selected. Comparing these summed values allows us to find which candidate has the largest count, meaning it appeared most frequently among all inputs.

A. Half Adder (HA)



A half adder is a simple circuit that adds two single binary bits and produces two outputs: a sum and a carry.

The Boolean expression of a half adder:

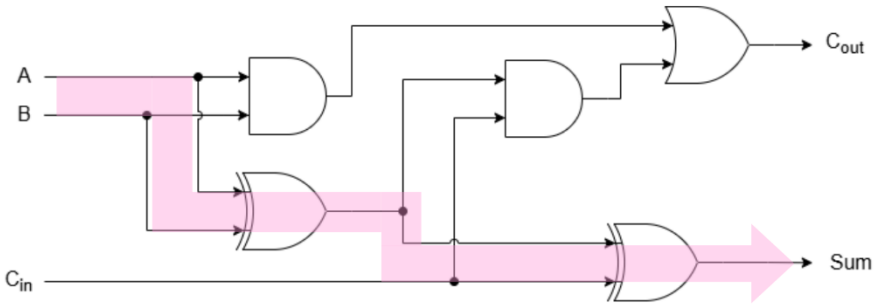
$$Sum = A \oplus B$$

$$C_{out} = A \cdot B$$

i. Logic Gates: 2

ii. Critical Path: 5Δ (as indicated by the pink arrow)

B. Full Adder (FA)



A full adder is a circuit that adds three binary bits, two input bits and an additional carry-in, and produces two outputs: a sum bit and a carry-out bit.

The Boolean expressions of a full adder:

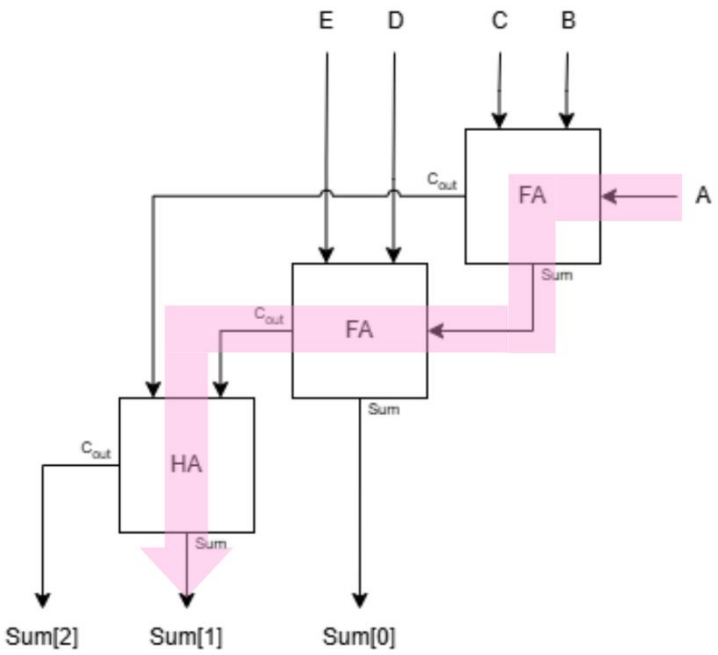
$$Sum = A \oplus B \oplus C_{in}$$

$$C_{out} = A \cdot B + C_{in} \cdot (A \oplus B)$$

i. Logic Gates: 5

ii. Critical path: 10Δ (as indicated by the pink arrow)

C. 5-Input Binary Adder



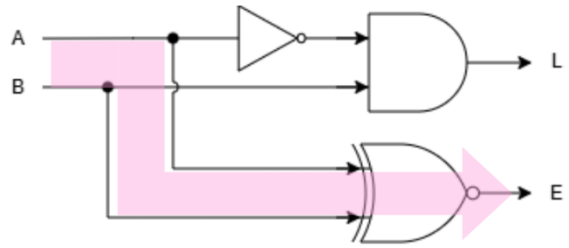
This circuit counts the number of 1s among five 1-bit inputs and outputs the total in binary using two full adders and one half adder. The first full adder (FA1) adds A, B, and C to generate a partial sum and a carry. The second full adder (FA2) then adds D, E, and the partial sum, producing the least significant bit Sum[0] and an intermediate carry. Finally, the half adder (HA) adds the two carry bits, generating Sum[1] and Sum[2]. Together, the three output bits {Sum[2], Sum[1], Sum[0]} represent the sum of A~E.

i. Logic Gates: $1 \times 2 + 2 \times 5 = 12$

ii. Critical Path: $10 + 9 + 5 = 24\Delta$

(as indicated by the pink arrow)

D. 1-bit Comparator



The design is a 1-bit comparator that determines the relationship between A and B. It outputs L when A is less than B, and outputs E when A and B are equal.

The Boolean expressions of a 1-bit comparator:

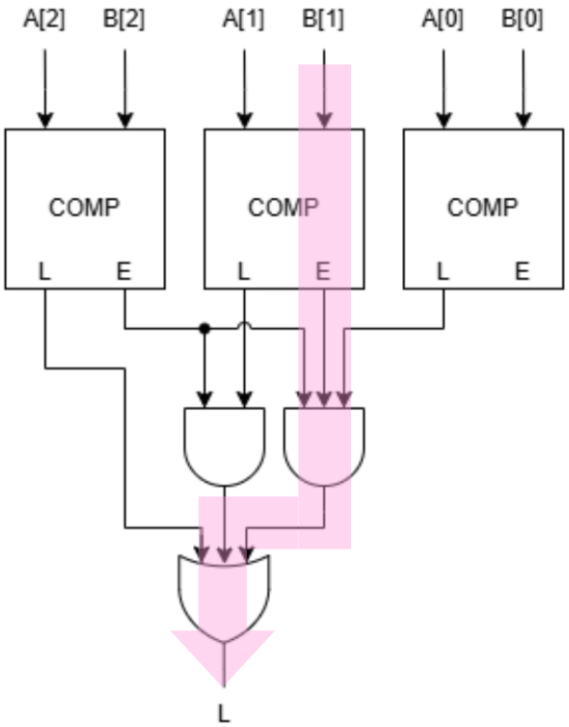
$$L = (\sim A) \cdot B$$

$$E = \sim(A \hat{} B)$$

i. Logic Gates: 3

ii. Critical Path: 6Δ (as indicated by the pink arrow)

E. 3-bit Comparator



The circuit compares two 3-bit numbers A and B and outputs L when A < B. It uses three 1-bit comparators and decides in MSB-first order.

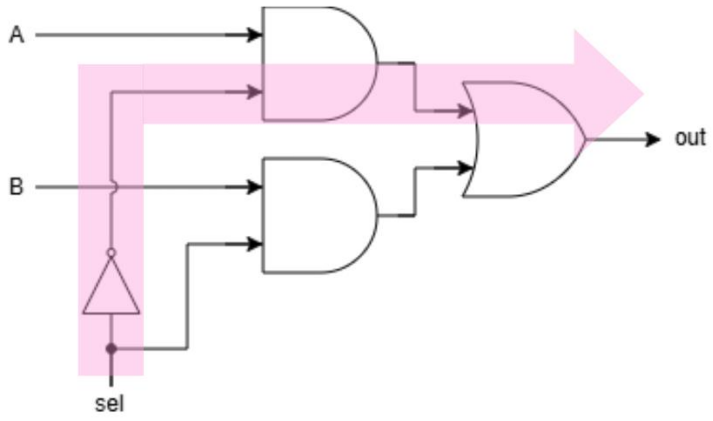
The Boolean expressions of a 3-bit comparator:

$$L = L[2] + E[2] \cdot L[1] + E[2] \cdot E[1] \cdot L[0]$$

i. Logic Gates: $3 \times 3 + 3 = 12$

ii. Critical Path: $6 + 2 + 2 = 10\Delta$
(as indicated by the pink arrow)

F. 1-bit 2-to-1 Multiplexer

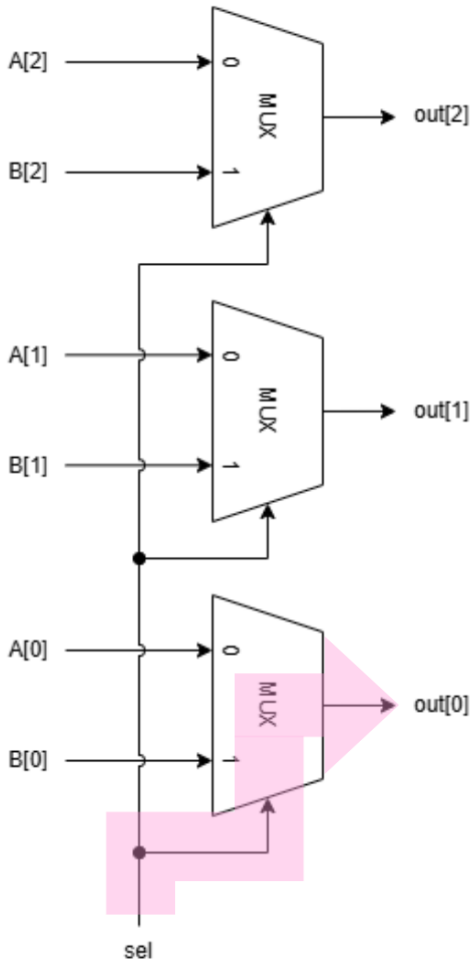


The circuit is a multiplexer that outputs A when sel=0 and outputs B when sel=1.

i. Logic Gates: 4

ii. Critical Path: 5Δ (as indicated by the pink arrow)

G. 3-bit 2-to-1 Multiplexer

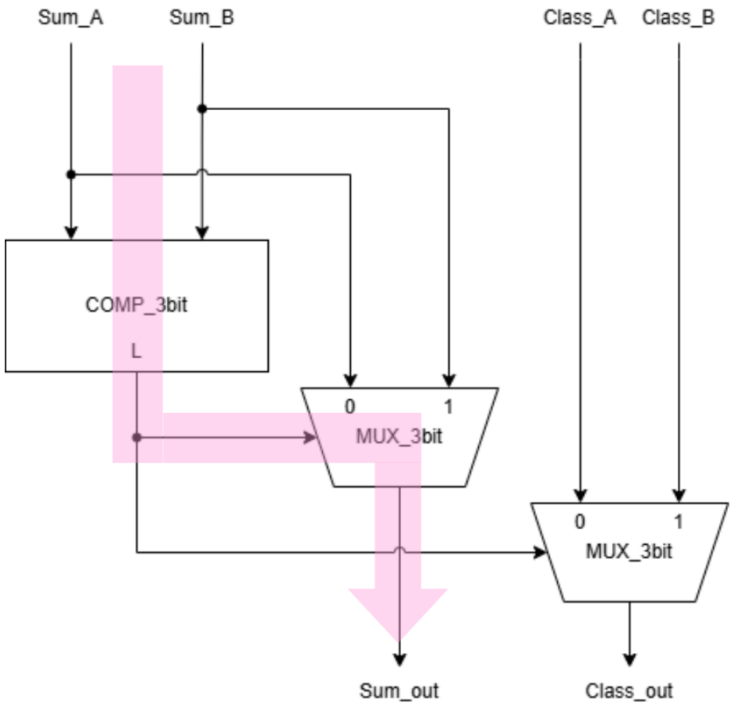


The circuit is a 3-bit multiplexer built from three 1-bit multiplexers in parallel. When sel=0 the output is A[2:0]; when sel=1 the output is B[2:0].

i. Logic Gates: $3 \times 4 = 12$

ii. Critical Path: 5Δ (as indicated by the pink arrow)

H. 3-bit 2-to-1 Selector

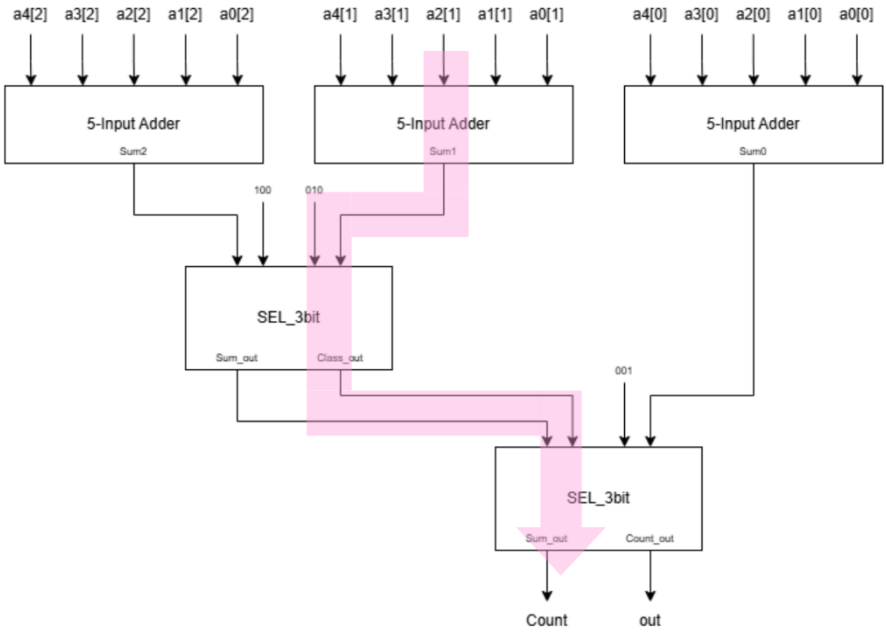


The circuit selects between two 3-bit data pairs, (class_A, sum_A) and (class_B, sum_B), based on the larger sum. The select signal, derived from the comparator's L output, goes high when sum_A < sum_B, causing the circuit to output class_B and sum_B; otherwise, it outputs class_A and sum_A, defaulting to A when the sums are equal.

i. Logic Gates: $1 \times 12 + 2 \times 12 = 36$

ii. Critical Path: 15Δ (as indicated by the pink arrow)

I. Voting Circuit



The voting circuit determines which bit position among five 3-bit one-hot inputs has the most logic-1 values. Each 5-input adder counts how many inputs are high in one bit position, producing three sums corresponding to the MSB, middle, and LSB. These sums are then compared in two stages using two 3-bit 2-to-1 selectors to find the maximum. The output out indicates the winning bit position, while count represents the number of votes it receives across the five inputs. By setting the comparison order, the circuit resolves ties by prioritizing the higher-significant bit, ensuring that the MSB is chosen when counts are equal.

i. Logic Gates: $3 \times 12 + 2 \times 36 = 108$

ii. Critical Path: $24 + 2 \times 15 = 54\Delta$

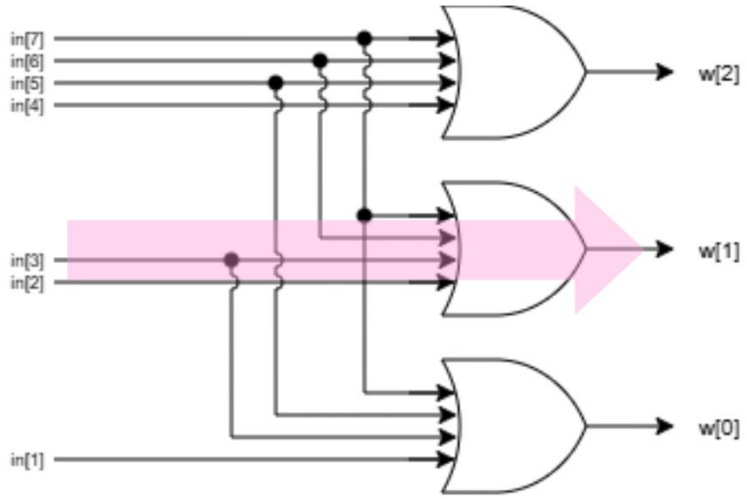
(as indicated by the pink arrow)

2. Median Circuit

The Median Circuit determines which of the three 8-bit one-hot inputs represents the median and outputs the corresponding value.

We can first convert each 8-bit one-hot input into a 3-bit binary number that indicates the position of its asserted bit. Then, three 3-bit comparators operate in parallel to evaluate the relationships between these values. The results of these comparisons form a vector that can be decoded to identify which input lies between the smallest and largest values. Finally, we can select the corresponding one-hot input as the median output.

A. One-Hot to Binary Encoder



The circuit takes an 8-bit one-hot input and produces a 3-bit binary output indicating the index of the asserted bit.

i. Logic Gates: 3

ii. Critical Path: 2Δ (as indicated by the pink arrow)

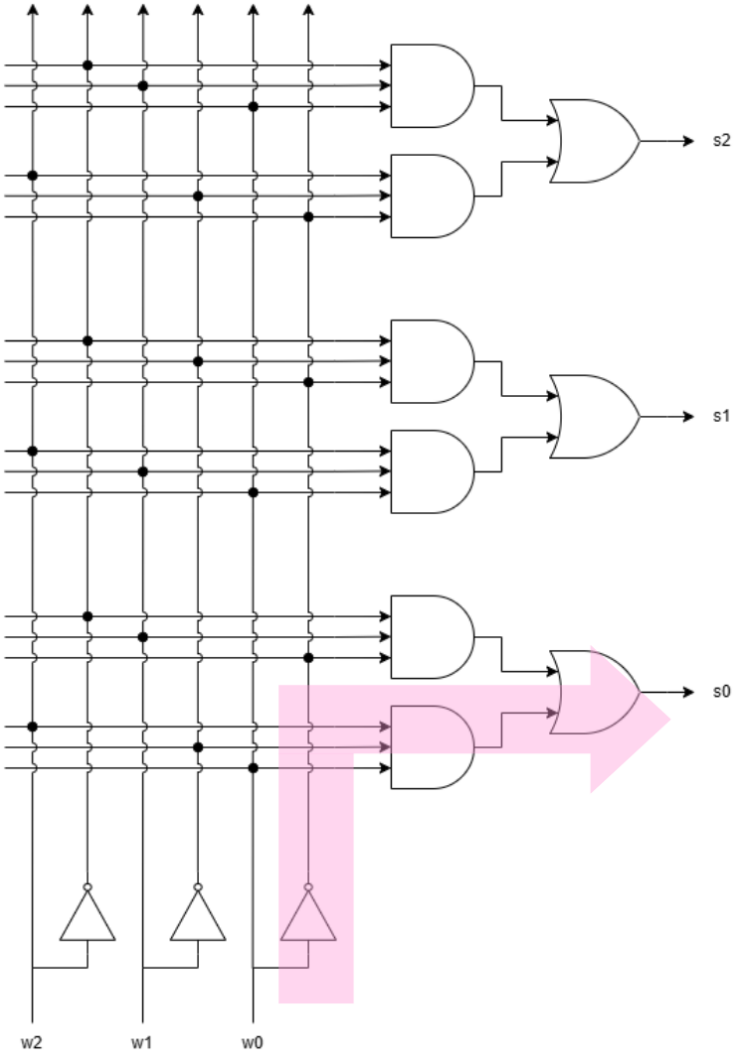
B. 3-bit Comparator

The circuit compares two 3-bit values and outputs a signal indicating whether input A is less than input B. It uses the same comparator structure as in the voting circuit.

i. Logic Gates: $3 \times 3 + 3 = 12$

ii. Critical Path: $6 + 2 + 2 = 10\Delta$

C. Median Selector



The median selector works together with the compare logic to determine which of the three one-hot inputs represents the median value.

The compare logic first generates a 3-bit vector w[2:0],

where each bit is the result of a 3-bit comparison:

$$w2 = A_2 < A_1$$

$$w1 = A_1 < A_0$$

$$w0 = A_2 < A_0$$

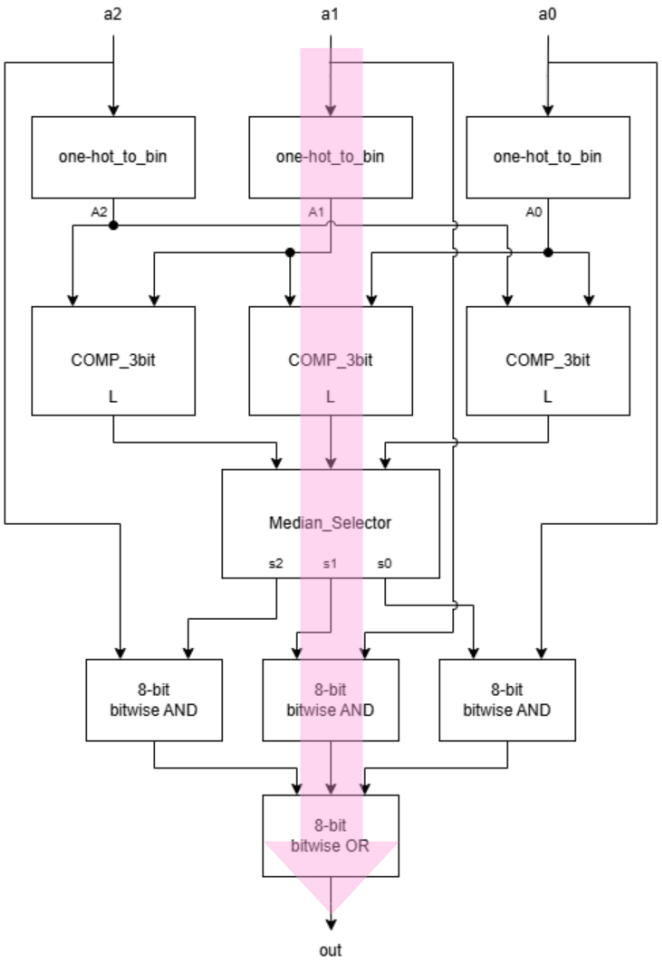
The median selector then decodes this comparison pattern into one-hot control signals s0, s1, and s2. Each signal corresponds to one of the inputs a0, a1, or a2, indicating which one should be chosen as the median.

The Boolean equations for s0, s1, and s2 cover the six valid combinations of w that can occur:

{w[2], w[1], w[0]}	Median
000	a1
001	Not Possible
010	a0
011	a2
100	a2
101	a0
110	Not Possible
111	a1

i. Logic Gates: 12
ii. Critical Path: 5Δ (as indicated by the pink arrow)

D. Median Circuit



This design finds the middle value among three 8-bit one-hot inputs with a combinational pipeline structure. First, each one-hot input is converted into a 3-bit binary value that represents the position of its asserted bit. Next, three comparators evaluate the ordering to rank the three values relative to one another. A selector then interprets that ordering and raises exactly one of three select signals to indicate which input is the median. Finally, an 8-bit bus multiplexer forwards the chosen one-hot input to out.

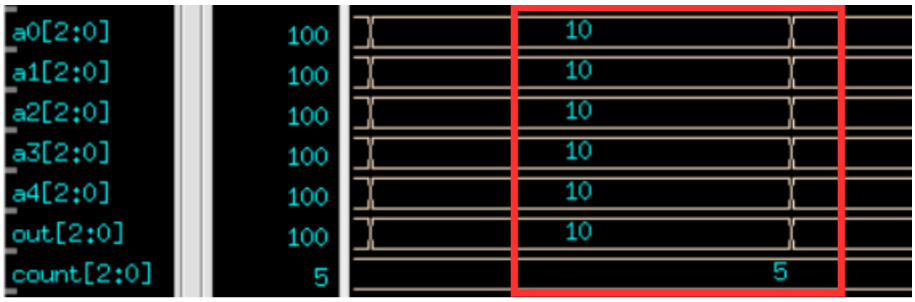
i. Logic Gates: $3 \times 3 + 3 \times 12 + 12 + 32 = 89$

ii. Critical Path: $2 + 10 + 5 + 2 + 2 = 21\Delta$
(as indicated by the pink arrow)

II. Result Analysis

1. Voting Circuit

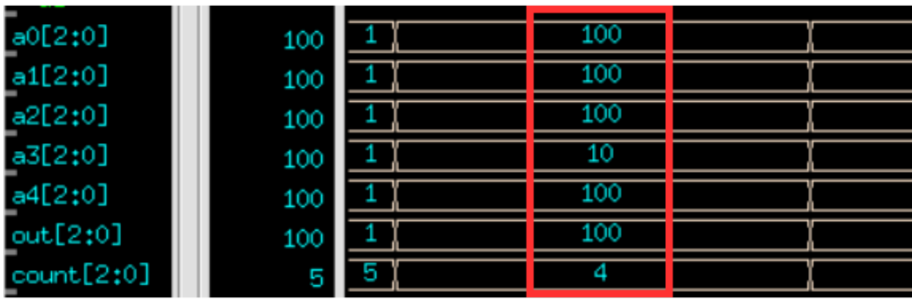
A. 5 Same



$100: 0 \quad 010: 5 \quad 001: 0$

$\rightarrow out = 010 \quad count = 5$

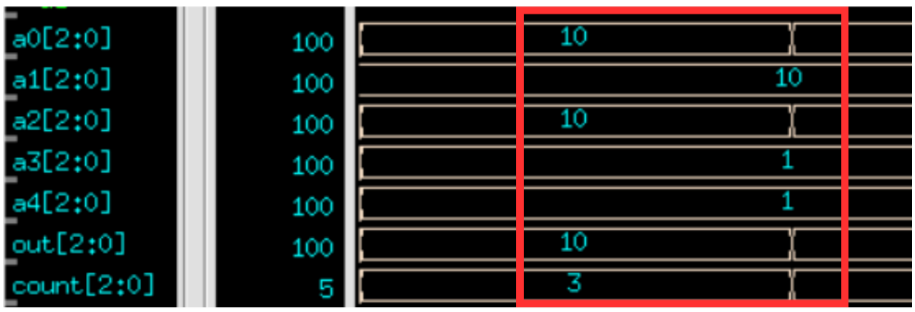
B. 4 Same + 1 Different



$100: 4 \quad 010: 1 \quad 001: 0$

$\rightarrow out = 100 \quad count = 4$

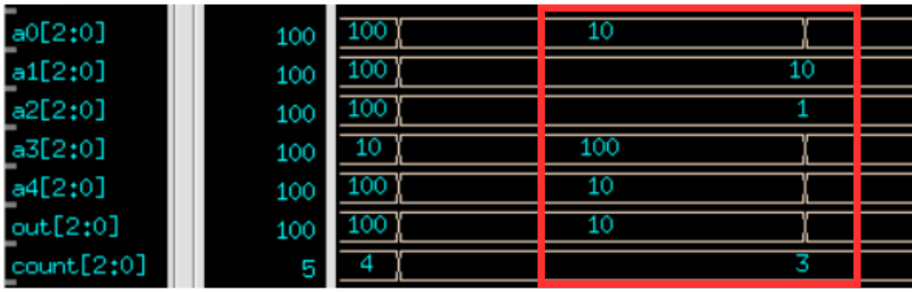
C. 3 Same + 2 Same



$100: 0 \quad 010: 3 \quad 001: 2$

$$\rightarrow out = 010 \qquad count = 3$$

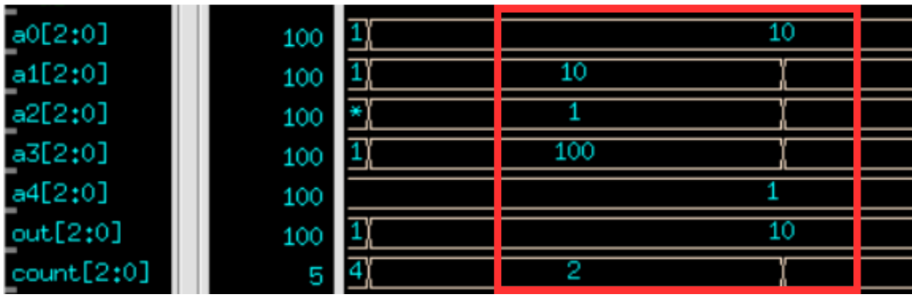
D. 3 Same + 2 Different



$$100: 1 \qquad 010: \ 3 \qquad 001: 1$$

$$\rightarrow out = 010 \qquad count = 3$$

E. 2 Same + 2 Same + 1 Different



$$100: 1 \qquad 010: \ 2 \qquad 001: 2$$

$$\rightarrow out = 010 \qquad count = 2$$

2. Median Circuit

A. 3 Same



$$00001000 \leq 00001000 \leq 00001000$$

$$\rightarrow out = 00001000$$

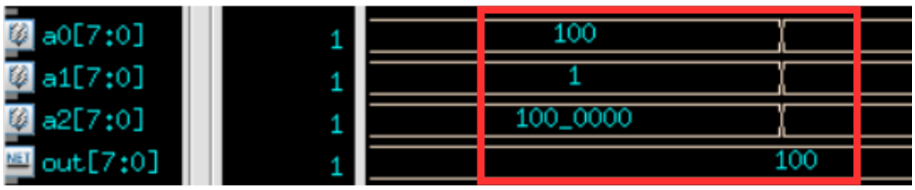
B. 2 Same + 1 Different



$$00000100 \leq 00000100 \leq 01000000$$

$$\rightarrow out = 00000100$$

C. 3 Different

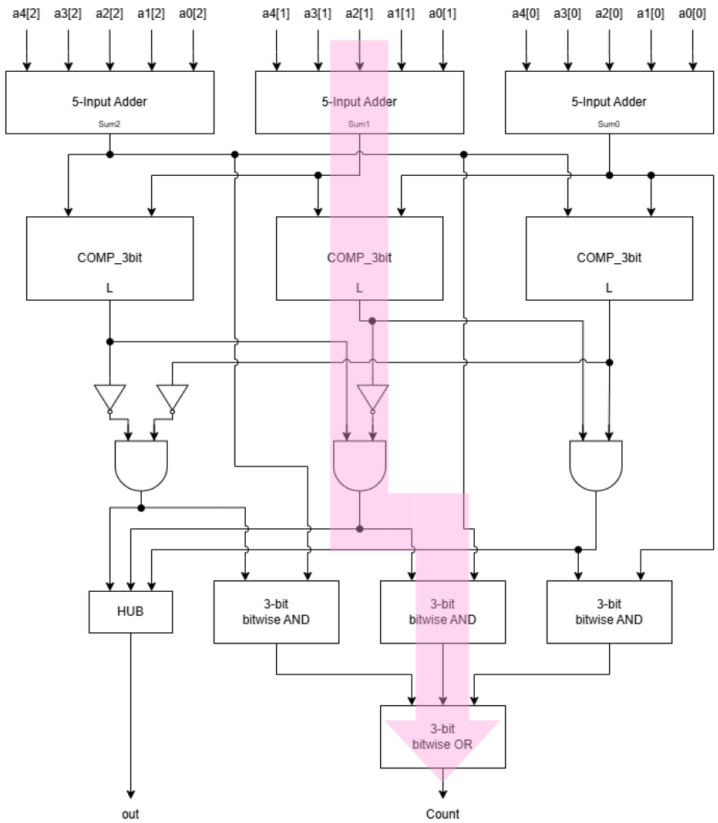


$$00000001 \leq 00000100 \leq 01000000$$

$$\rightarrow out = 00000100$$

III. Alternative Design

1. Voting Circuit



Instead of sequentially comparing pairs of sums using cascaded 3-bit selectors, three 3-bit comparators operate concurrently to evaluate all pairwise relationships among the partial sums.

$$w[2] = a_2 < a_1$$

$$w[1] = a_1 < a_0$$

$$w[0] = a_2 < a_0$$

The results of these comparisons form a 3-bit comparison vector w[2:0], which is then decoded by simple Boolean logic to generate a one-hot output signal out[2:0] indicating the bit position with the highest vote count.

$$out[0] = w[1] \cdot w[0]$$

$$out[1] = w[2] \cdot (\sim w[1])$$

$$out[2] = (\sim w[2]) \cdot (\sim w[0])$$

Once the winning position is determined, each bit of out can directly enable its corresponding sum through simple AND–OR gating.

$$count = (out[0] \cdot sum0) + (out[1] \cdot sum1) + (out[2] \cdot sum2)$$

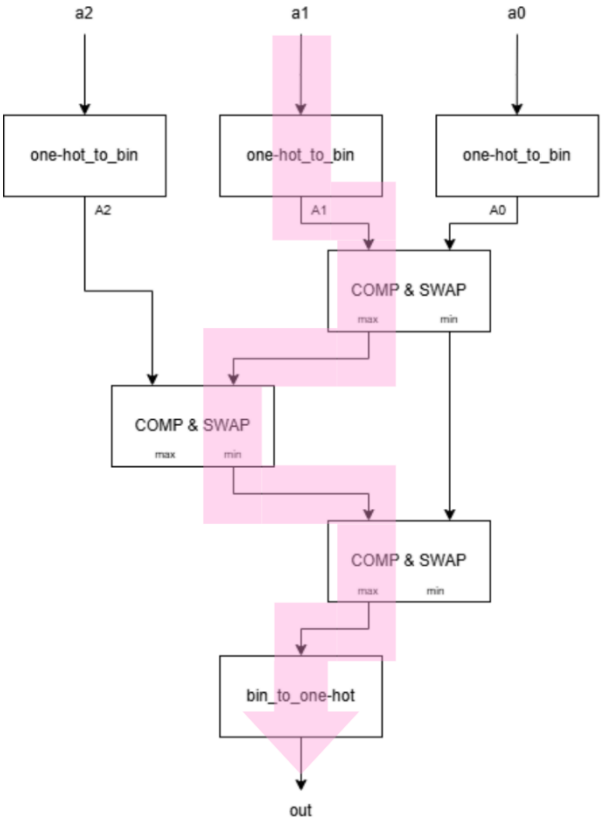
This structure eliminates one level of comparator and multiplexer delay present in the sequential version, thereby shortening the critical path and reducing the area.

i. Logic Gates: $3 \times 12 + 3 \times 12 + 6 + 12 = 90$

ii. Critical Path: $24 + 10 + 7 = 41\Delta$

(as indicated by the pink arrow)

2. Median Circuit



Similar to the previous design, each input is first converted into a 3-bit binary index using the encoder. However, instead of performing all pairwise comparisons simultaneously and decoding the results, this version incrementally orders the three indices through a sequence of compare-and-swap operations. Each compare-and-swap module consists of a 3-bit comparator and two 3-bit 2-to-1 multiplexers, which output the smaller value as min and the larger as max. Finally, the result is then converted back into the one-hot format by a decoder, producing the 8-bit output.

i. Logic Gates: $3 \times 3 + 3 \times 36 + 11 = 128$

ii. Critical Path: $2 + 3 \times 15 + 3 = 50\Delta$

(as indicated by the pink arrow)

IV. Comparison

1. Voting Circuit

	Version 1	Version 2
Algorithm	Cascaded Selections	Parallel Compares
Logic Gates	Higher	Lower
Critical Path	Longer	Shorter
Performance	Worse	Better

2. Median Circuit

	Version 1	Version 2
Algorithm	Parallel Compares	Compare & Swap
Logic Gates	Lower	Higher
Critical Path	Shorter	Longer
Performance	Better	Worse

V. Testbench Design

In both testbenches, the golden model serves as a reference to verify the DUT outputs automatically. For the voting circuit, it counts how many times each bit position (001, 010, and 100) appears among the five one-hot inputs, producing counts c0, c1, and c2. The model then compares these counts and applies the defined tie-break priority (100 > 010 > 001) to determine the expected winning output and its corresponding count. For the median circuit, the golden model converts each 8-bit one-hot input into its index position (0~7), finds the smallest and largest indices, and computes the median using the equation $median = x + y + z - min - max$. The resulting median

index is then converted back into an 8-bit one-hot value for direct comparison with the DUT output. Both perform directed tests for edge and tie cases, followed by random tests for broader coverage. After all simulations, an ALL PASS banner is displayed when no errors are found; otherwise, detailed failure messages are printed for debugging.

VI. Discussion & Reflection

Through this homework, I learned about how much a good circuit design can significantly affect both speed and area. Even small changes in structure, like how the comparators are arranged or whether we use multiplexers or simple logic, can make a big difference. In the voting circuit, redesigning it from a two-stage selector chain into a parallel comparison with one-hot gating made the whole system faster and simpler, since it removed extra layers of logic. On the other hand, the second version of the median circuit, which used a compare-and-swap network, looked more systematic but ended up using more gates and having a much longer delay because of the extra stages. Through these experiments, I realized that circuit design is all about trade-offs, sometimes we sacrifice area for clarity, or complexity for speed. Overall, this assignment helped me understand that good designs aren’t just about making something work, but about finding smarter ways to make it more efficient.